CODE:

```
// ------------------------------------------------------------
// 1. C1 extends C2
// A class can extend another class.
// ------------------------------------------------------------

class C2 {
   void methodC2() {
      System.out.println("Method of C2");
   }
}

class C1 extends C2 {
   void methodC1() {
      System.out.println("Method of C1");
   }
}

// ------------------------------------------------------------
// 2. C1 extends C2, C3
//  NOT VALID IN JAVA
// A class cannot extend two classes at the same time.
/*
class C3 {
   void methodC3() {
      System.out.println("Method of C3");
    }
```

```
 class C1 extends C2, C3 { }  // ERROR
*/
// Java does not support multiple inheritance using classes.
// ------------------------------------------------------------


// ------------------------------------------------------------
// 3. C1 implements I1
// A class can implement an interface.
// ------------------------------------------------------------

interface I1 {
   void methodI1();
}

// C1 already extends C2, and it can also implement I1.
class C1WithInterface extends C2 implements I1 {

   public void methodI1() {
      System.out.println("Method of I1");
   }
}

// ------------------------------------------------------------
// 4. C1 implements I1, I2
// A class can implement multiple interfaces.
// ------------------------------------------------------------
```

```
interface I2 {
   void methodI2();
}

class C3 implements I1, I2 {

   public void methodI1() {
      System.out.println("I1 method implemented");
   }

   public void methodI2() {
      System.out.println("I2 method implemented");
   }
}

// ------------------------------------------------------------
// 5. C1 implements C2 extends I1
// NOT VALID
//
// C2 is a class and I1 is an interface.
// Correct syntax is:
//
// class C1 extends C2 implements I1
//
// Example:

class C4 extends C2 implements I1 {   //Error
```

```
    public void methodI1() {
        System.out.println("C4 implements I1");
    }
}

// ---------------------------------------------------------

// ---------------------------------------------------------
// 6. I1 extends I2
// An interface can extend another interface.
// ---------------------------------------------------------

interface I4 {
    void methodI4();
}

interface I5 extends I4 {
    void methodI5();
}

// ---------------------------------------------------------
// 7. I1 implements C1
// NOT VALID
//
// An interface cannot implement a class.
//
// interface I6 implements C2 { }  // ERROR
//
```

```
// An interface can only EXTEND another interface.
// ------------------------------------------------------------


// ------------------------------------------------------------
// 8. I1 extends I2, I3
// An interface can extend multiple interfaces.
// ------------------------------------------------------------

interface I7 {
    void methodI7();
}

interface I8 {
    void methodI8();
}

// I9 extends two interfaces
interface I9 extends I7, I8 {
    void methodI9();
}

// ------------------------------------------------------------
// MAIN CLASS
// ------------------------------------------------------------

public class Main {

    public static void main(String[] args) {
```

```
// 1. C1 extends C2
C1 obj1 = new C1();

obj1.methodC2();  // inherited from C2
obj1.methodC1();  // own method

System.out.println("-----------------------------");

// 3. C1 implements I1
C1WithInterface obj2 = new C1WithInterface();

obj2.methodC2();  // inherited from C2
obj2.methodI1();  // implemented from I1

System.out.println("-----------------------------");

// 4. C1 implements I1, I2
C3 obj3 = new C3();

obj3.methodI1();  // from I1
obj3.methodI2();  // from I2

System.out.println("-----------------------------");

// 5. Correct form:
// C1 extends C2 implements I1
```

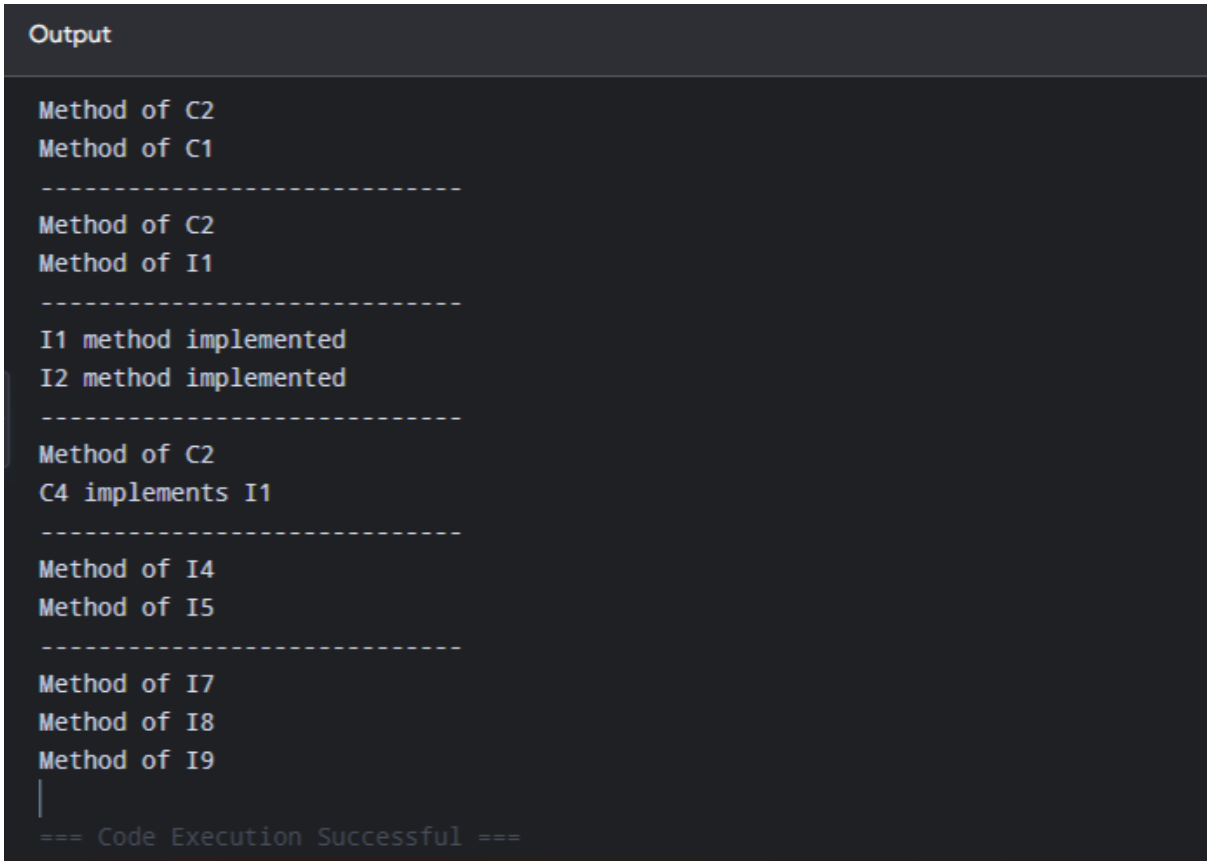
```
C4 obj4 = new C4();

obj4.methodC2();  // inherited from C2
obj4.methodI1();  // implemented from I1

System.out.println("----------------------------");

// 6. I1 extends I2
// I5 extends I4.
// Therefore, a class implementing I5 must implement
// methods from both I5 and I4.

I5 obj5 = new I5() {

    public void methodI4() {
        System.out.println("Method of I4");
    }

    public void methodI5() {
        System.out.println("Method of I5");
    }
};

obj5.methodI4();
obj5.methodI5();

System.out.println("----------------------------");
```

```
        // 8. I1 extends I2, I3
        // I9 extends both I7 and I8.

        I9 obj6 = new I9() {

            public void methodI7() {
                System.out.println("Method of I7");
            }

            public void methodI8() {
                System.out.println("Method of I8");
            }

            public void methodI9() {
                System.out.println("Method of I9");
            }
        };

        obj6.methodI7();
        obj6.methodI8();
        obj6.methodI9();
    }
}
```

EXPLAINATION:

1.C1 extends C2

class C2 {




```
void methodC2() { System.out.println("Method of C2"); }
}
class C1 extends C2 {
void methodC1() { System.out.println("Method of C1"); }
}
```

- C2 is the parent class and C1 is the child class.
- The keyword extends makes C1 inherit C2.
- Therefore a C1 object can call methodC2() and methodC1().

2. C1 extends C2, C3:

- // class C1 extends C2, C3 { } // ERROR
- This is invalid.

- Java does not allow a class to extend two classes. Multiple inheritance through classes is not supported.

3. C1 implements I1:

```
interface I1 {
void methodI1();
}
class C1WithInterface extends C2 implements I1 {
public void methodI1() {
System.out.println("Method of I1");
}
}
```

- A class can implement an interface.
- C1WithInterface extends C2 and implements I1, so it inherits C2 and provides the required implementation of methodI1().

4. C1 implements I1, I2:

```
class C3 implements I1, I2 {
public void methodI1() { System.out.println("I1 method implemented"); }
public void methodI2() { System.out.println("I2 method implemented"); }
}
```

- A class can implement multiple interfaces.
- C3 implements both I1 and I2, so it must implement the methods declared by both interfaces.

5. C1 implements C2 extends I1:

```
// class C4 implements C2 extends I1 { } // ERROR
```

- This is invalid. C2 is a class, so it cannot follow implements.
- The correct syntax is class C4 extends C2 implements I1. Also, extends must come before implements.

6. I1 extends I2:

```
interface I4 {
void methodI4();
}
interface I5 extends I4 {
void methodI5();
}
```

- An interface can extend another interface.
- I5 inherits the method declaration from I4 and also declares methodI5().
- A class implementing I5 must implement both methods.

7. I1 implements C1:

```
// interface I6 implements C2 { } // ERROR
```

- This is invalid.
- An interface cannot implement a class.
- A class implements an interface, while an interface extends another interface.

8. I1 extends I2, I3:

```
interface I7 { void methodI7(); }
interface I8 { void methodI8(); }
interface I9 extends I7, I8 {
void methodI9();
}
```

- This is valid.
- An interface can extend multiple interfaces.
- I9 gets method declarations from both I7 and I8 and adds methodI9().

9. Main and Objects:

```
C1 obj1 = new C1();
C1WithInterface obj2 = new C1WithInterface();
C3 obj3 = new C3();
C4 obj4 = new C4();
```

- These statements create objects.
- Each object is used to call its own, inherited, or implemented methods.

OUTPUT:

Output

```
Method of C2
Method of C1
------------------------------
Method of C2
Method of I1
------------------------------
I1 method implemented
I2 method implemented
------------------------------
Method of C2
C4 implements I1
------------------------------
Method of I4
Method of I5
------------------------------
Method of I7
Method of I8
Method of I9

=== Code Execution Successful ===
```